Linux Filesystem

1. What is FileSystem?

A file system is a method used by the operating system to organize, store, and manage data on a storage device like a hard disk. It defines how data is stored, accessed, and retrieved

2. Windows vs Linux

Windows path: C:\Users\<user_name>\Downloads

Linux: /home/<user_name>/<file_name>

Windows is case insensitive and Linux is case sensitive

3. Types of File Systems (Common)

ext4 → most common in Linux
xfs → used in enterprise systems
ntfs → Windows
fat32 → USB drives

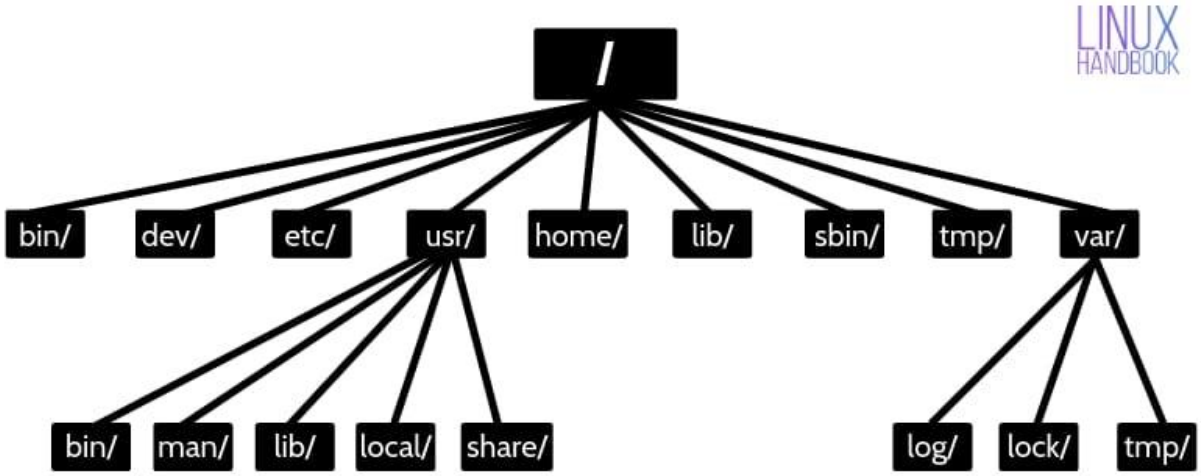
4. Difference between ext4 and XFS

ext4 and XFS are both Linux file systems. ext4 is more general-purpose and widely used, while XFS is designed for high performance and large-scale systems, especially for handling large files and parallel operations.

Key Differences

Feature	ext4	xfs
Type	General purpose	High-performance
Performance	Good	Better for large data
File Size Support	Up to ~16TB	Very large (petabytes)
Resize	Can shrink & grow	Only grow (cannot shrink)
Speed	Moderate	Faster for large files
Journaling	Yes	Yes (more efficient)
Use Case	Default Linux FS	Enterprise / Big data

5. Linux File System Structure



/

Root directory — the top of the filesystem; everything starts from here.

/bin

Essential user commands (like ls, cp, mv) required for basic system operation.

/sbin

System/admin commands used for system maintenance (mainly for root user).

/etc

Stores system-wide configuration files (users, services, network configs).

/home

Contains personal directories for all regular users.

/root

Home directory of the root (admin) user.

/var

Stores variable data like logs, cache, mail, and application data.

/tmp

Temporary files; usually cleared automatically after reboot.

/usr

Contains user-installed applications, libraries, and documentation.

/opt

Used for installing third-party or optional software.

/dev

Contains device files representing hardware (disks, USB, etc.).

/proc

Virtual filesystem showing process and system information (CPU, memory, running processes).

/boot

Contains bootloader files and Linux kernel required for system startup.

/mnt

Temporary mount point for manually mounted filesystems.

/media

Auto-mounted removable media like USB drives and CDs.

/lib & /lib64

Shared libraries required by system programs and binaries.

Users & Groups

1. What is a User?

A **user** is a person or system account that can log into Linux and perform actions.

Example:	**Login Command**
• Root -> admin user	sudo su
• Steve -> normal user	su - <user_name>

2. What is a Group?

A **group** is a collection of users.

Uses:

- To manage permission easily.
- Instead of giving access to each user, you give access to a group.

Example:

- Group: Developers
- Users: ram, shyam , alex

Commands:

```
useradd john      # create user
adduser john      # advanced mode
groupadd dev      # create group
userdel john      #delete user
groupdel dev      #delete group
usermod -aG dev john   # add user to group
gpasswd –d john dev  #remove john from dev group
```

id <user_name>

output: uid=1001(user_name) gid=1004(group_name) groups=1004(group_name),1001(dev)

User ID | **Group ID**

File Permissions

File Permissions (rwx)

Every file/directory has permissions that define: Who can read, write, or execute it

Three Types of Permissions:

Symbol	Meaning	Example
r	Read	View file
w	Write	Edit file
x	Execute	Run file

Three Levels of Access:

Level	Meaning
User (u)	Owner of file
Group (g)	Group members
Others (o)	Everyone else

Each permission has a number:

Permission	Value
r (read)	4
w (write)	2
x (execute)	1

Example1:

-rwxr-xr--

Section	**Meaning**
Rwx	Owner → full access
r-x	Group → read & execute
r--	Others → read only

Easy Trick: Always read in groups of 3:

rwx | r-x | r--

u g o

<u>Change Persmission:</u>

chmod 755 file.txt

7 → rwx

5 → r-x

5 → r-x

<u>Example2:</u>

mkdir /tmp/shared

chmod 1777 /tmp/shared

1 777

| └── Normal permissions (rwx for all)

└─── Special permission (sticky bit)

Owner → full access

Group → full access

Others → full access

So everyone can read, write, create files

What is that 1? (Sticky Bit)

1 = Sticky Bit

What Sticky Bit does

Rule:

Users can create files, but can only delete their own files

<u>Example3:</u>

chmod g+s /tmp/project

This command sets the SGID (Set Group ID) on the /tmp/project directory.

What SGID does?

When SGID is applied on a directory:

All new files and folders created inside will inherit the same group as the directory.

Without SGID

/project → group = dev

User john creates file:

file.txt → group = john

Problem:

Group mismatch

Team cannot access properly

With SGID (After chmod g+s)

/project → group = dev

User john creates file:

file.txt → group = dev

Solution:

All team members (dev group) can access

Ownership

Every file has:

- Owner (user)
- Group

Example1:

-rw-r--r-- 1 john dev file.txt

- Owner = john
- Group = dev

Change Ownership

- chown john file.txt **# change owner**
- chown john:dev file.txt **# change owner + group**

Example2:

-rwxr----- 1 john dev script.sh

- Owner (john) → full access
- Group (dev) → read only
- Others → no access

Example3:

- chown john:dev file.txt
- chmod 770 file.txt
- Owner + group → full access
- Others → no access

Users = who is using system

Groups = collection of users

Permissions = what they can do

Ownership = who controls the file

#Recursive flag: It inherits the command property.

Ex: file path: /home/ubuntu/file.txt

chmod 755 /home/ubuntu #it will change permission for only ubuntu dir

chmod -R 755 /home/ubuntu #it will change permission for ubuntu along with all files & dirs inside ubuntu.

-R → recursive (safe meaning across commands, applicable everywher)

-r → depends on command (cannot use in chmod, chown)

Process Lifecycle

How a process lives & dies

1. What is a Process?

A **process** is equal to a running program

Example:
sleep 100 #command
This creates a process

2. Lifecycle stages

A process goes through different states:

New → Ready → Running → Waiting → Terminated

a. New (Created)

Process is created

Example: you type a command

b. Ready

Waiting for CPU

In queue

c. Running

CPU is executing it

d. Waiting (Sleeping)

Waiting for I/O (disk, network, etc.)

Example:

sleep 10 #command

It is in **waiting state**

e. Terminated

Process finished or killed

How to check process states

ps aux #command

Look at column:

- R → Running
- S→ Sleeping
- Z → Zombie (important)

3. What is Zombie process?

-> Process is dead but still in table

-> Parent didn’t read its status

Kill a process

kill <PID> #command

kill -9 <PID> # force kill command

Foreground vs Background Process

1. Foreground process

-> Runs in terminal and blocks it

Example:

sleep 20 #command

You cannot type anything until it finishes

2. Background process

-> Runs in background → terminal is free

Example:

sleep 100 & #command

Output:

[1] 1234

-> 1 → job id

-> 1234 → PID

Move running process to background

a. Run:

sleep 100 #command

b. Press:

Ctrl + Z

Suspends process

c. Then:

bg #command

Runs in background

Bring back to foreground

fg #command

See background jobs

jobs #command

Summary

Type	Behavior
Foreground	Blocks terminal
Background	Runs freely

System Monitoring (Very Important)

1. Why monitoring?

To check:

-> CPU usage

-> Memory usage

-> Running processes

-> System load

a. **Check CPU usage**

i. top #command

Shows:

-> CPU %
-> Memory
-> Running processes

ii. Better version

htop #command

More user-friendly (install if not present)

b. **Check memory**

free -h #command

Output:

-> total

-> used

-> free

c. **Check processes**

ps aux #command

d. Check specific process

ps aux | grep nginx #command

e. System load

uptime #command

Shows load average

f. Disk usage

df –h #command

g. Real-time process tracking

top –p <PID> #command

htop Command explanation:

1. CPU Colors Explanation

Color	Meaning	Good/Bad
Green	User processes	✅ Normal
Red	Kernel/system	⚠️ Watch
Blue	Nice processes	✅ Fine
Yellow	I/O wait	❌ Disk issue
Purple	Steal time	❌ VM issue
Grey	Idle	✅ Good

Case 1: Mostly Green

-> Normal application load
✓ Healthy system

Case 2: High Red

Kernel busy
Possible causes:

-> Heavy I/O
-> Network issues
-> Driver problem

Case 3: High Yellow

Disk is slow
Common in:
-> Database servers
-> Logging-heavy systems

Case 4: High Purple

Cloud issue

-> Shared CPU overloaded

2. Memory Colors Explained

Green → Used Memory (Applications)
Memory used by:
-> Running programs (Java, Python, etc.)
-> Services (nginx, mysql)
This **is real usage**

Blue → Buffers
Memory used by kernel for:
-> Temporary data
-> Disk operations
✓ Small portion usually
✓ Helps system performance

Yellow → Cache

Memory used for:

-> File caching

-> Frequently accessed data

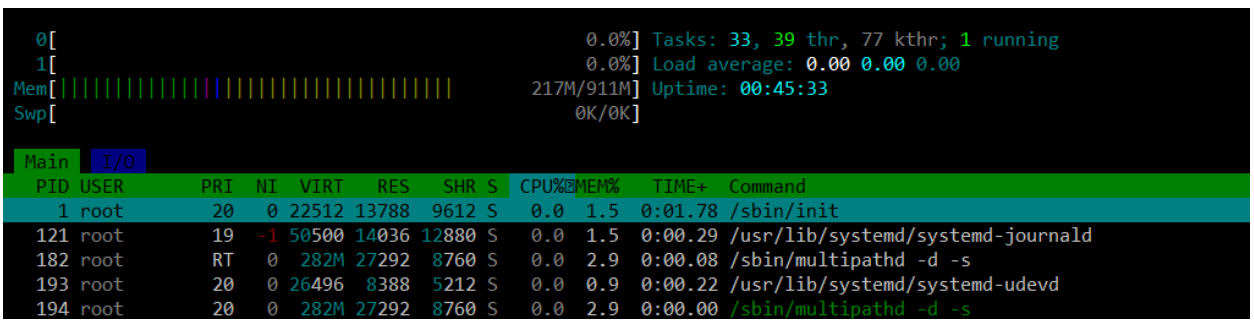
✓ VERY IMPORTANT:

This memory can be freed anytime if needed

Grey → **Free Memory**

-> Completely unused RAM

Color	Meaning	Important?
Green	App memory	✅ Yes
Blue	Buffers	Minor
Yellow	Cache	✅ Very important
Grey	Free	Not critical



a. PID: Process ID

b. User: Name of users

c. PRI(Priority): Lower number -> Higher priority, kernel decides this based on NI

d. NI (Nice value): controls priority manually. Ex: -20(highest priority) , 19(Lowest Priority), 0 (Default Priority)

e. VIRT(Virtual Memory): Total memory used by process including everything (RAM, SWAP, Shared lib, reserved memory). *Note: VIRT is often large and misleading.*

f. RES (Resident memory): Actual RAM used right now.

g. SHR (Shared Memory): Memory shared with other processes. Ex: shared libraries, common resources.

h. S (State): Process state.

top command explanation:

```
top - 11:48:58 up  1:02,  3 users,  load average: 0.00, 0.00, 0.00
Tasks: 116 total,   1 running, 115 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.2 us,  0.3 sy,  0.0 ni, 99.5 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :    911.5 total,    271.6 free,    382.3 used,    416.1 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    529.2 avail Mem
```

i) **11:48:58** -> current system time

ii) **up 1:02min** -> system running since last reboot

iii) **3 users** -> 2 logged-in users

iv) **load average** -> system load.

0.00, 0.00, 0.00 -> Last 1 min, 5 min, 15 min resp.

Ex: 0.00 -> no load (idle system), 1.00 -> CPU fully used, > CPU cores -> overloaded.

v) **Tasks:**

```
Tasks: 116 total,   1 running, 115 sleeping,   0 stopped,   0 zombie
```

Field	Meaning
total	total processes
running	currently using CPU
sleeping	waiting (normal)
stopped	paused (Ctrl+Z)
zombie	dead but not cleaned

Currently 1 running -> very light load.

115 sleeping -> normal

0 zombie -> healthy system

vi) **CPU usage:**

```
%Cpu(s):  0.2 us,  0.3 sy,  0.0 ni, 99.5 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
```

current -> 99.5 id CPU is 99.5% idlle

Field	Meaning
us	User processes
sy	System (kernel)
ni	Nice (low priority)
id	Idle
wa	Waiting for I/O
hi	Hardware interrupts
si	Software interrupts
st	Stolen (VM usage)

vii) **Memory (RAM):**

```
MiB Mem :    911.5 total,    271.6 free,    382.3 used,    416.1 buff/cache
```

buff/cache is not wasted. Linux is using free RAM as cache to improve performance.

Field	Meaning
total	total RAM
free	unused RAM
used	actively used
buff/cache	cached memory

viii) **Swap:**

```
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    529.2 avail Mem
```

disk used as RAM (backup memory.)
current -> No swap configured, available memory for swap: 592.2 MB